

Machine Learning for Physics Final Project

Abstract

In time-resolved spectroscopy, the stability of the optical pulse is an important prerequisite for reliable measurements. However, in fiber-optic systems, the pulse position may drift as a function of both time and temperature. The relationship between pulse position and temperature is not straightforward, suggesting that pulse drift may occur over time even when temperature has no change. If the pulse position can be accurately predicted or retrieved, the experimental alignment and data acquisition process could be made more efficient, thereby reducing the time required for repeated measurements. Therefore, several machine learning methods are applied in this study to investigate whether the pulse position can be predicted from the available experimental variables.

Methodology

The objective is to predict the pulse position x using time t and temperature T as input variables. Since x is a continuous variable, the task is formulated as a supervised regression problem. Two methods learnt in class, K-Nearest Neighbors Regression (KNN) and Multilayer Perceptron (MLP), will be applied. In addition, Polynomial Regression, Support Vector Regression, and Random Forest Regression will be used for comparison.

KNN is included as a simple non-parametric regression method that predicts x based on nearby samples in the (t, T) feature space. MLP is used because it can learn nonlinear mappings between the input variables and the target variable. Polynomial Regression is selected as an interpretable baseline, especially because physical motion often contains nonlinear dependence on time and possible interaction between time and temperature. SVR is used to capture nonlinear relationships through kernel functions, while Random Forest Regression is included as an ensemble method that can model complex interactions without assuming a specific functional form.

Before training the models, the dataset will be divided into training and testing sets, each containing 85 data. The training set will be used to fit the models, while the testing set will be used to evaluate their predictive performance on unseen data.

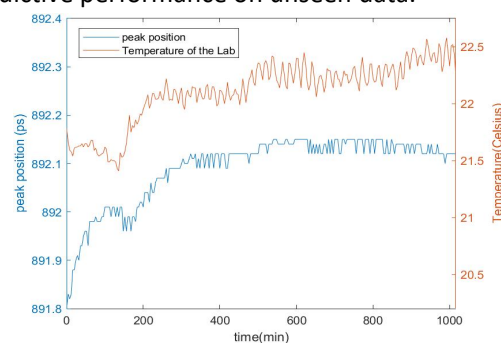


Figure 1. Pulse position shift in experiment.

Result

To begin with, the KNN model is evaluated. The value of k is tested over the range from 1 to 21. Based on the validation set, the optimal value is selected as $k=2$, which is then applied to the test set. The corresponding result is shown in Figure 1. For the test set, the model generally provides accurate predictions after $t=200$, with the absolute residual remaining below 0.04 and an MSE of 0.00046423. However, relatively large residuals are still observed at the early stage.

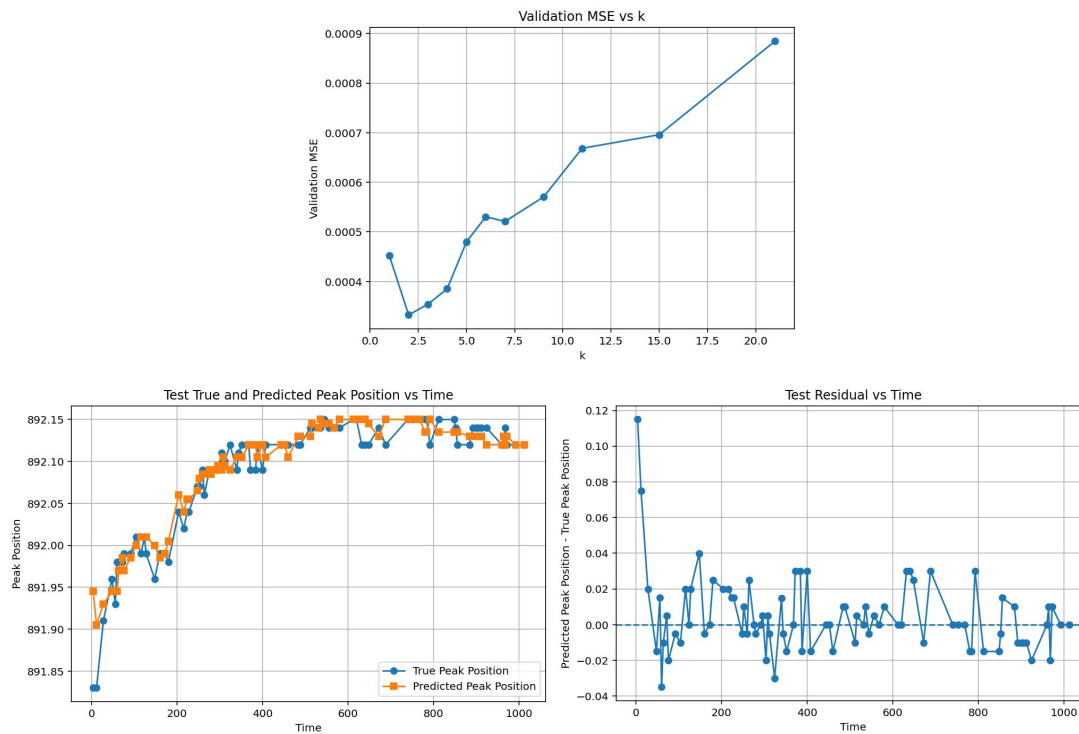


Figure 1. The results of KNN model.

Next, the MLP model is applied to the same dataset. Compared with KNN, the MLP appears to provide better predictions at the early stage, although its performance becomes less accurate at larger time values. Nevertheless, as shown in Figure 2, the residual generally remains below 0.04. The MSE of the MLP model is 0.00037113, indicating better prediction over the KNN model.

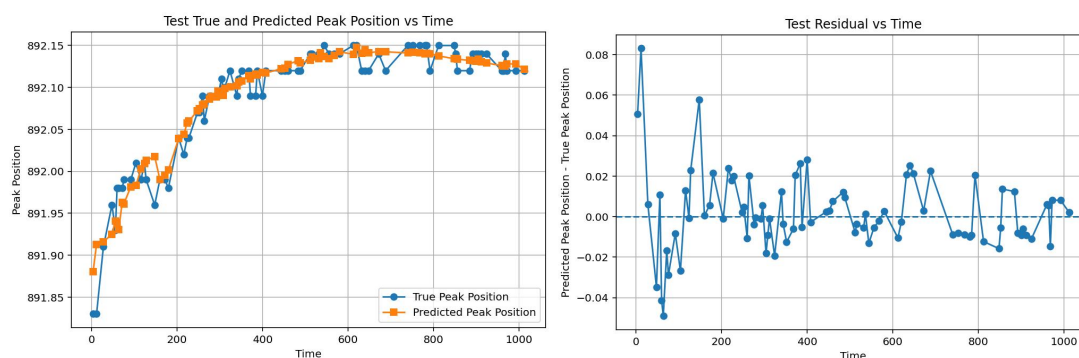


Figure 2. The results of MLP model.

Beyond the KNN and MLP models learned in class, polynomial regression is also tested to examine whether an explicit fitted equation can describe how time and temperature affect the

pulse position. Polynomial degrees from 1 to 5 are evaluated, and the third-order polynomial gives the best performance based on the validation set. The fitted equation is:

$$\text{peak_position} = 892.1341552734 + 0.0376064740 * t - 0.0034311537 * T - 0.0444717966 * t^2 - 0.0102693001 * t * T + 0.0066931085 * T^2 + 0.0056106979 * t^3 + 0.0537733249 * t^2 * T - 0.0684945658 * t * T^2 + 0.0219896361 * T^3.$$

Moreover, this model provides good predictions at the early stage, which is important for retrieving the pulse position when it moves fast. The results are shown in Figure 3. The final MSE is 0.00041084.

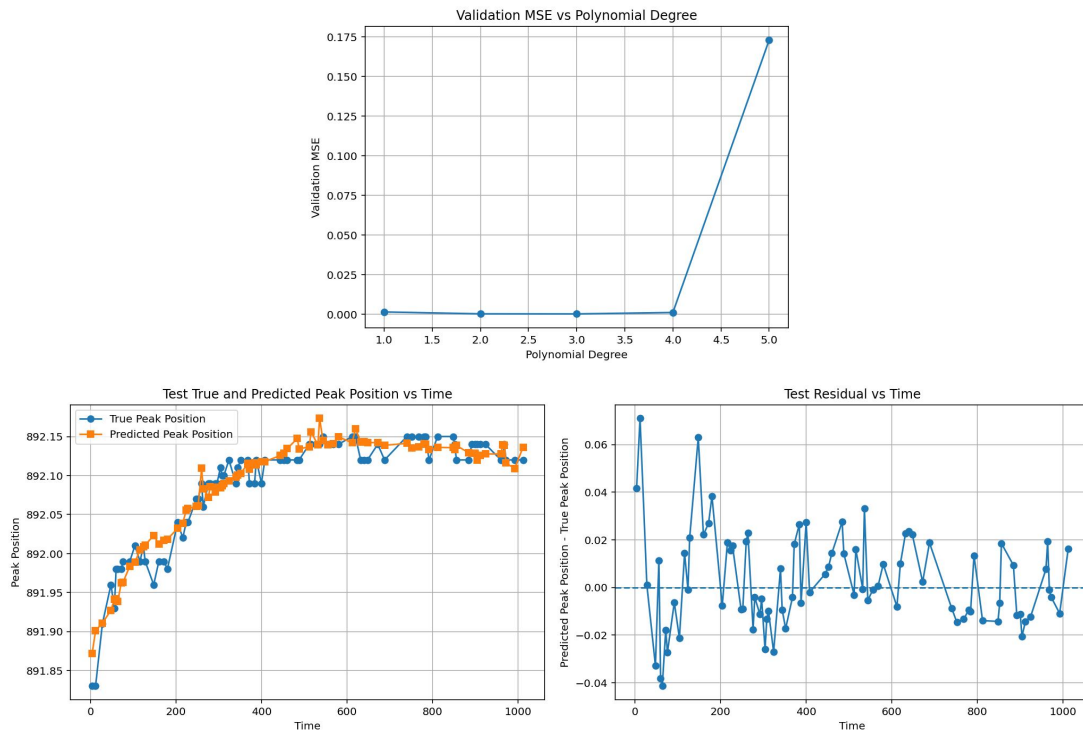


Figure 3. The results of polynomial regression.

Another method applied is SVR. Since standard SVR is usually implemented using the sklearn toolbox, this study compares the results obtained from sklearn SVR and a PyTorch SVR-style model. The sklearn SVR performs slightly better than the PyTorch version, yet the difference is negligible. The MSE values of the sklearn SVR and PyTorch SVR-style models are 0.00027770 and 0.00028900, respectively. In addition, SVR provides better predictions when the pulse position drift rapidly.

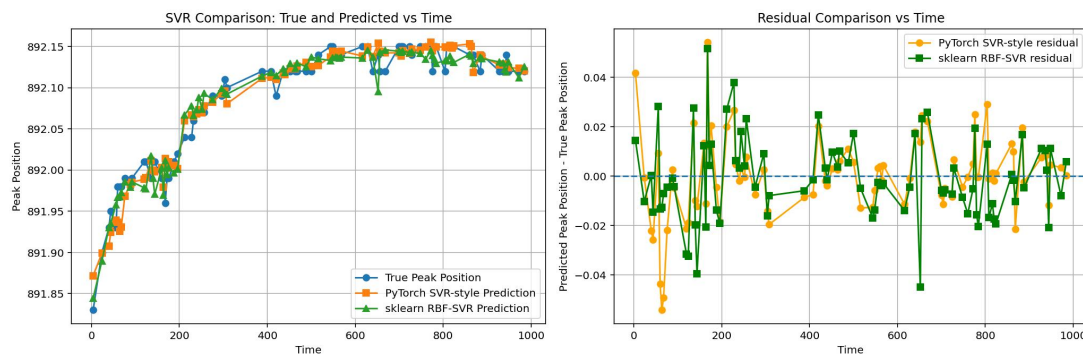


Figure 4. The comparison of results of sklearn and pytorch SVR.

The last model tested is Random Forest Regression, which estimates the relative importance of time and temperature in predicting the pulse peak position. As shown in Figure 5, time has a much higher feature importance than temperature. This suggests that the pulse shift cannot be explained only by temperature variation over time. Instead, time itself remains an important predictive variable, supporting the initial assumption that the pulse position is mainly affected by temporal evolution. The MSE of RFR is 0.00021647.

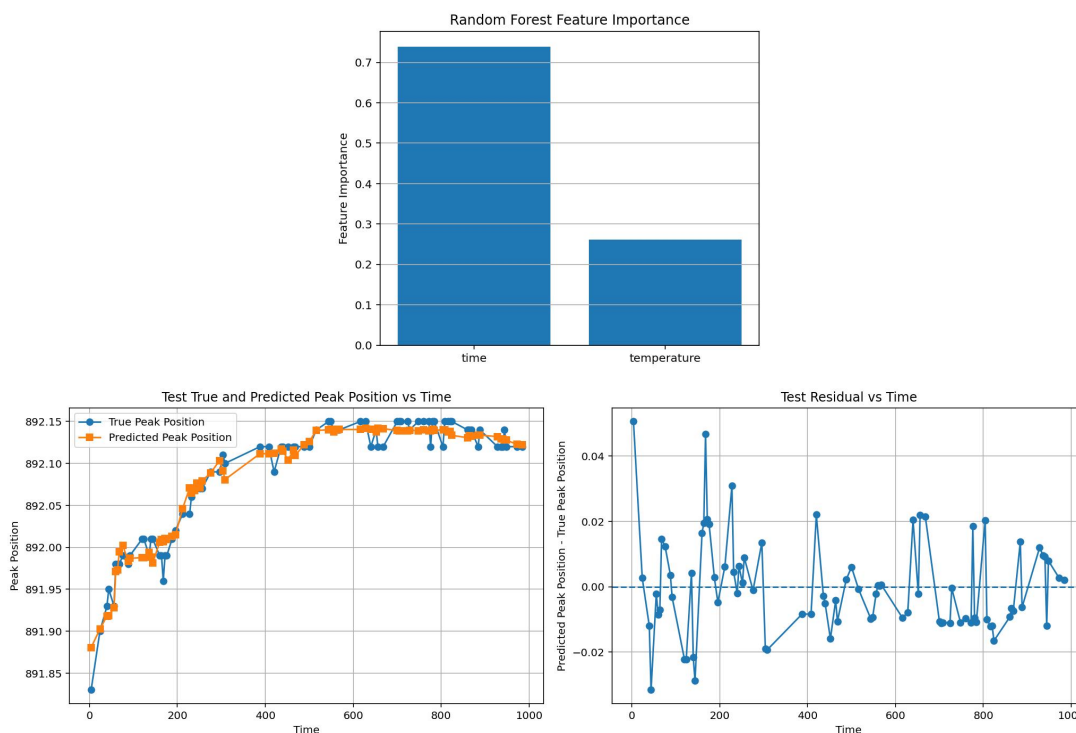


Figure 5. The results of random forest regression.

Conclusion

The behavior of pulse shift is successfully predicted using different machine learning models. Among these models, Random Forest Regression achieves the best performance, giving the lowest MSE on the test set. In addition, polynomial regression provides an explicit fitted equation, which can be used to estimate the pulse position in a more interpretable form.

These prediction models could be useful during experiments for retrieving the reference pulse position when direct tracking is difficult. Therefore, this approach may provide a practical solution for compensating pulse-position drift, especially when the experimental setup is unstable.

Link for Code

The codes for this project can be found at:

[https://github.com/YigYang/Machine-Learning-in-Physics/tree/main/Final Project](https://github.com/YigYang/Machine-Learning-in-Physics/tree/main/Final%20Project)